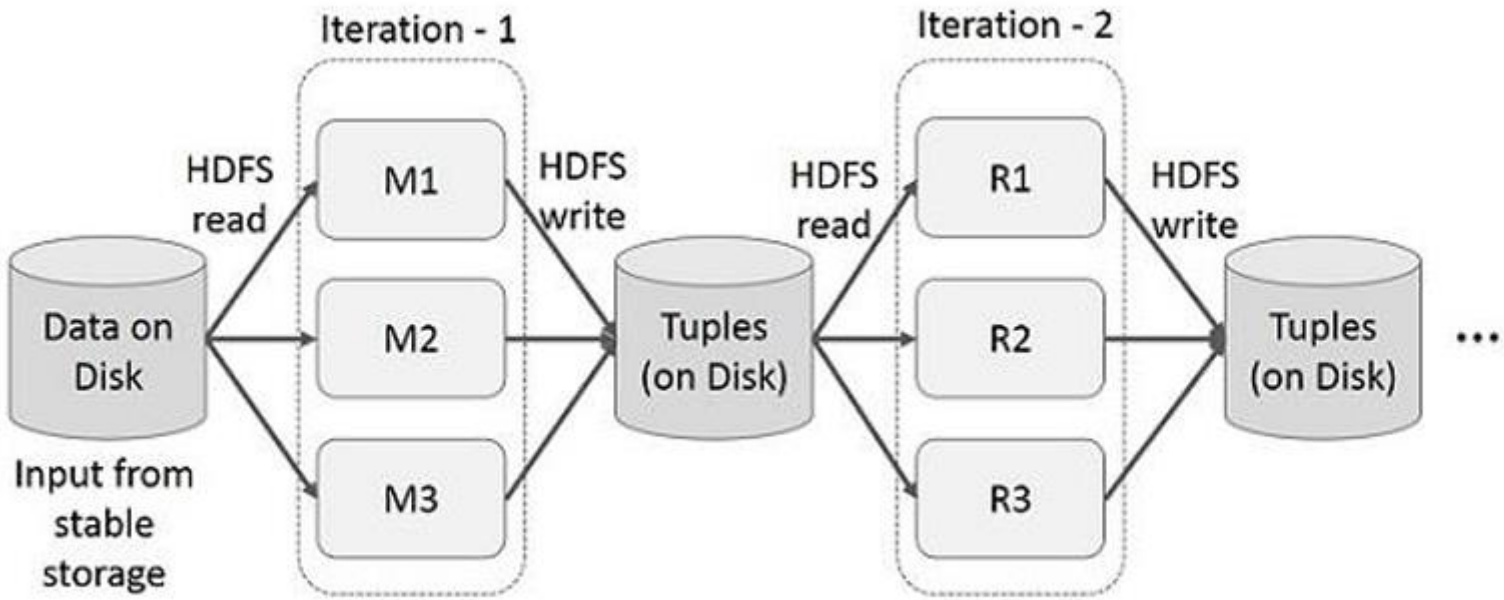
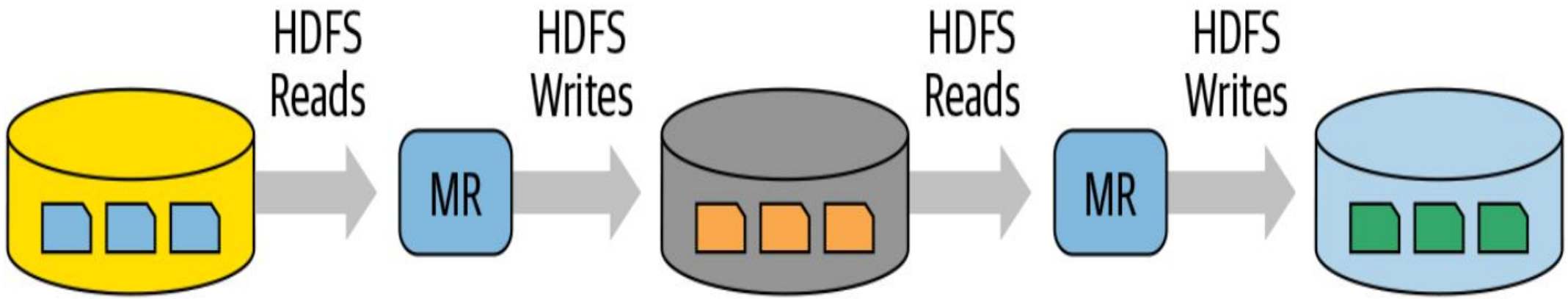
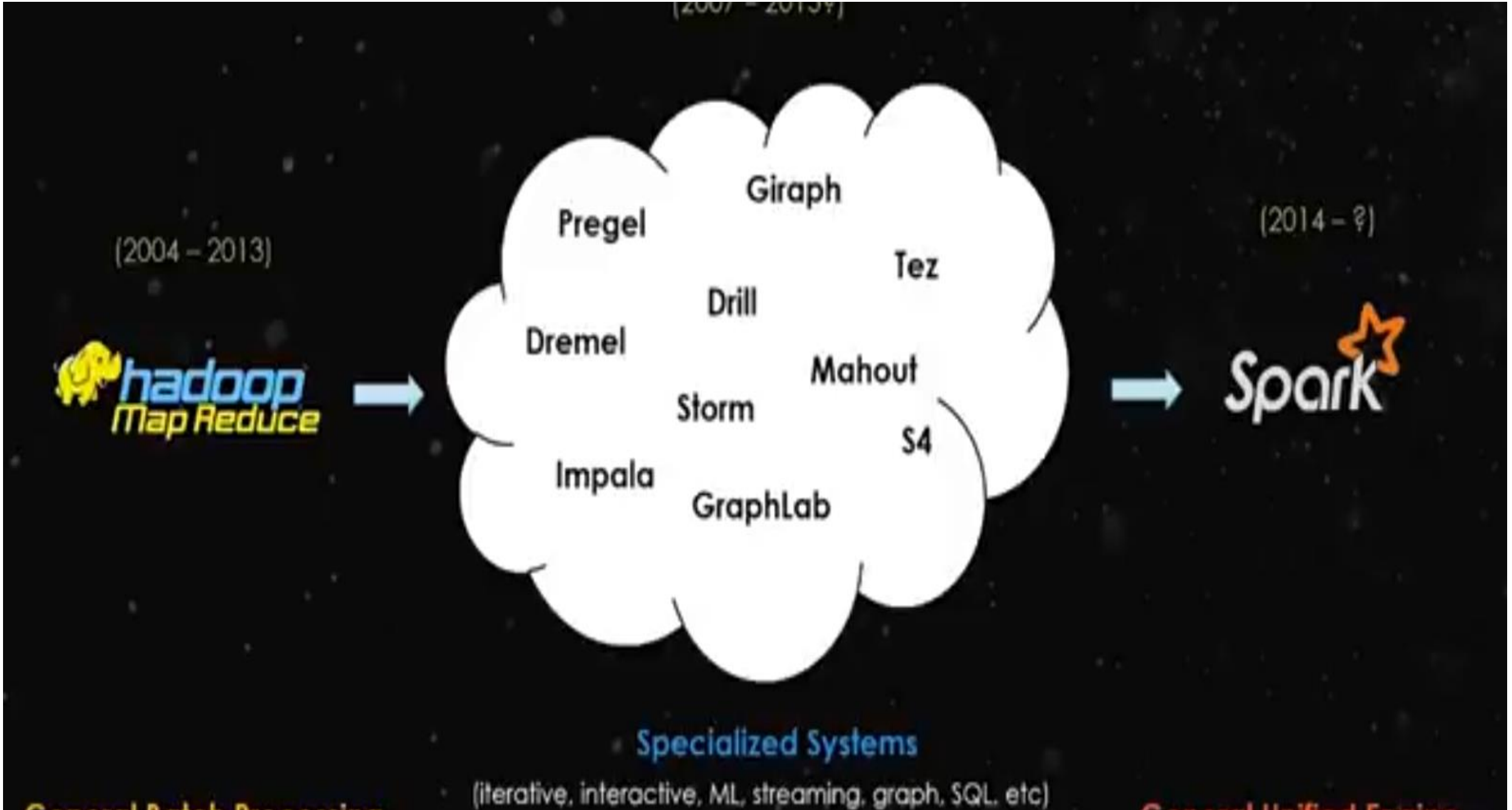
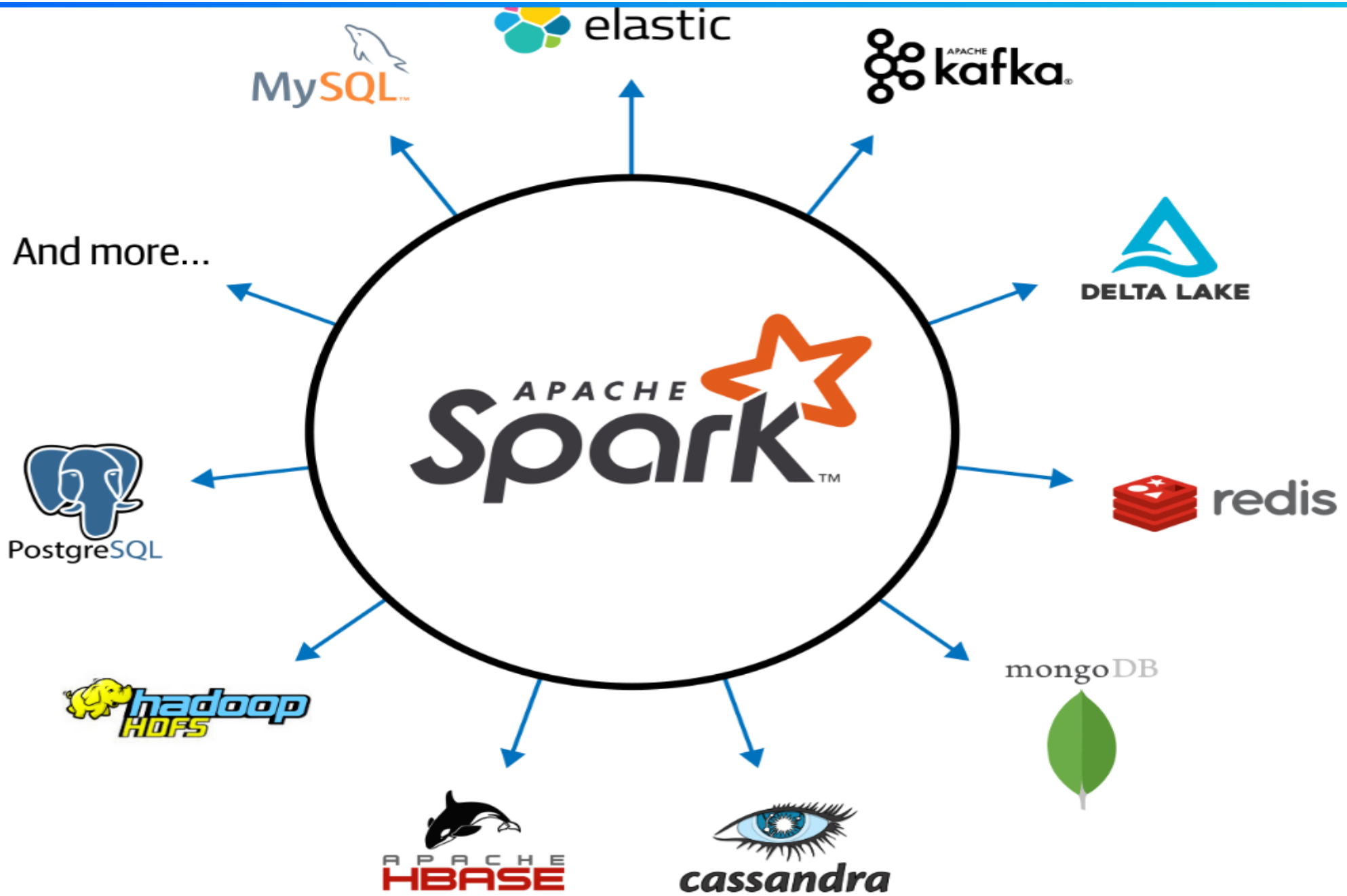


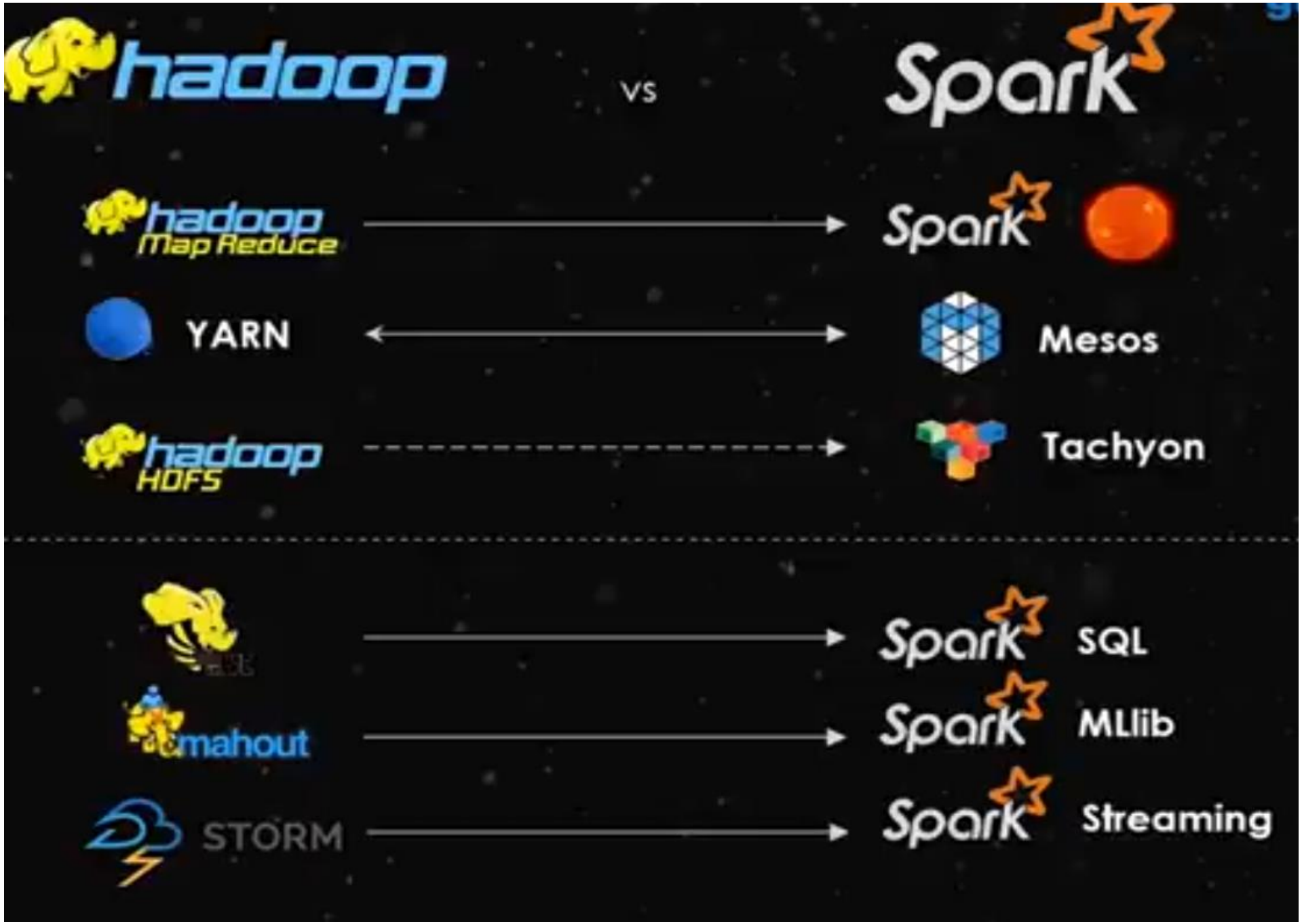
Figure: Iterative operations on MapReduce

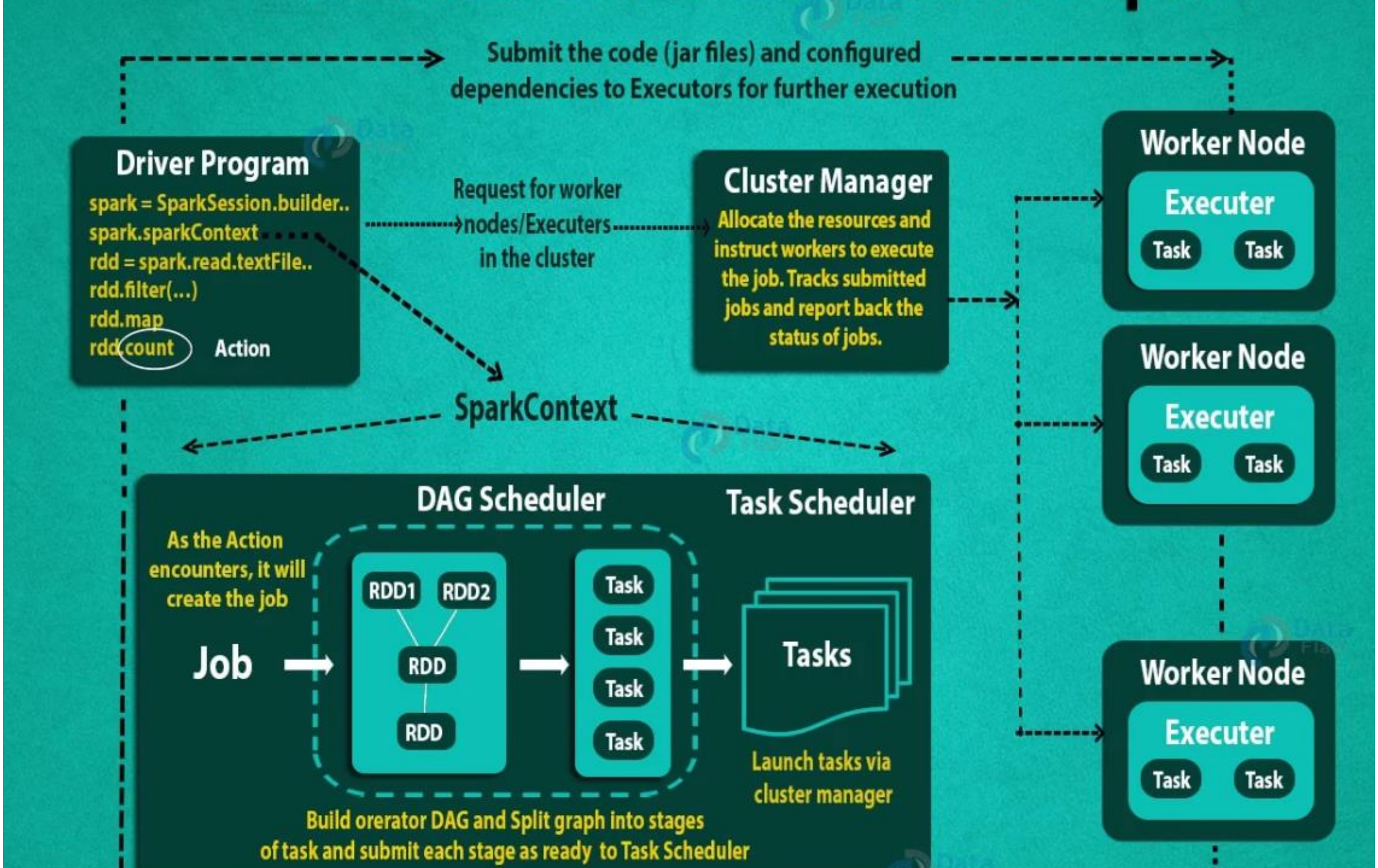

HDFS
Reads
MR
HDFS
Writes
HDFS
Reads
MR
HDFS
Writes


(2004 – 2013)
hadoop
Map Reduce
Giraph
Pregel
Tez
Drill
Dremel
Mahout
Storm
S4
Impala
GraphLab
(2014 – ?)
Spark
Specialized Systems
(iterative, interactive, ML, streaming, graph, SQL, etc)

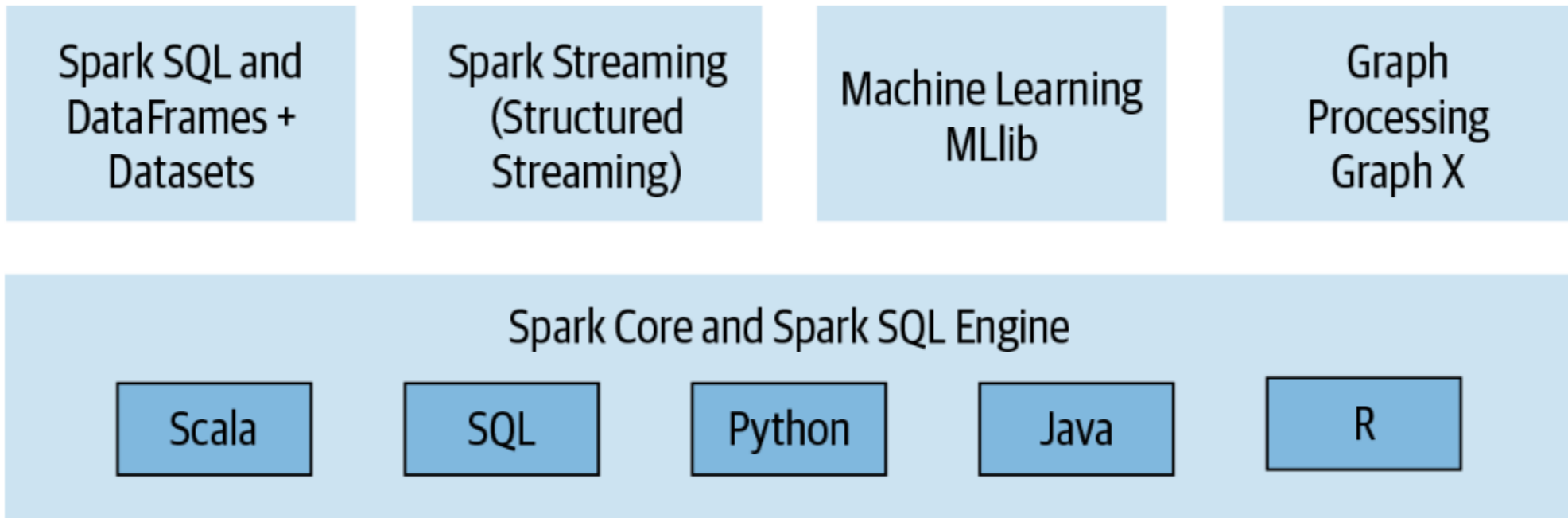

hadoop
HDFS
mongoDB
RDBMS
Neo4j
APACHE
HBASE
FILE://
SQL
Streaming
GraphX
MLlib

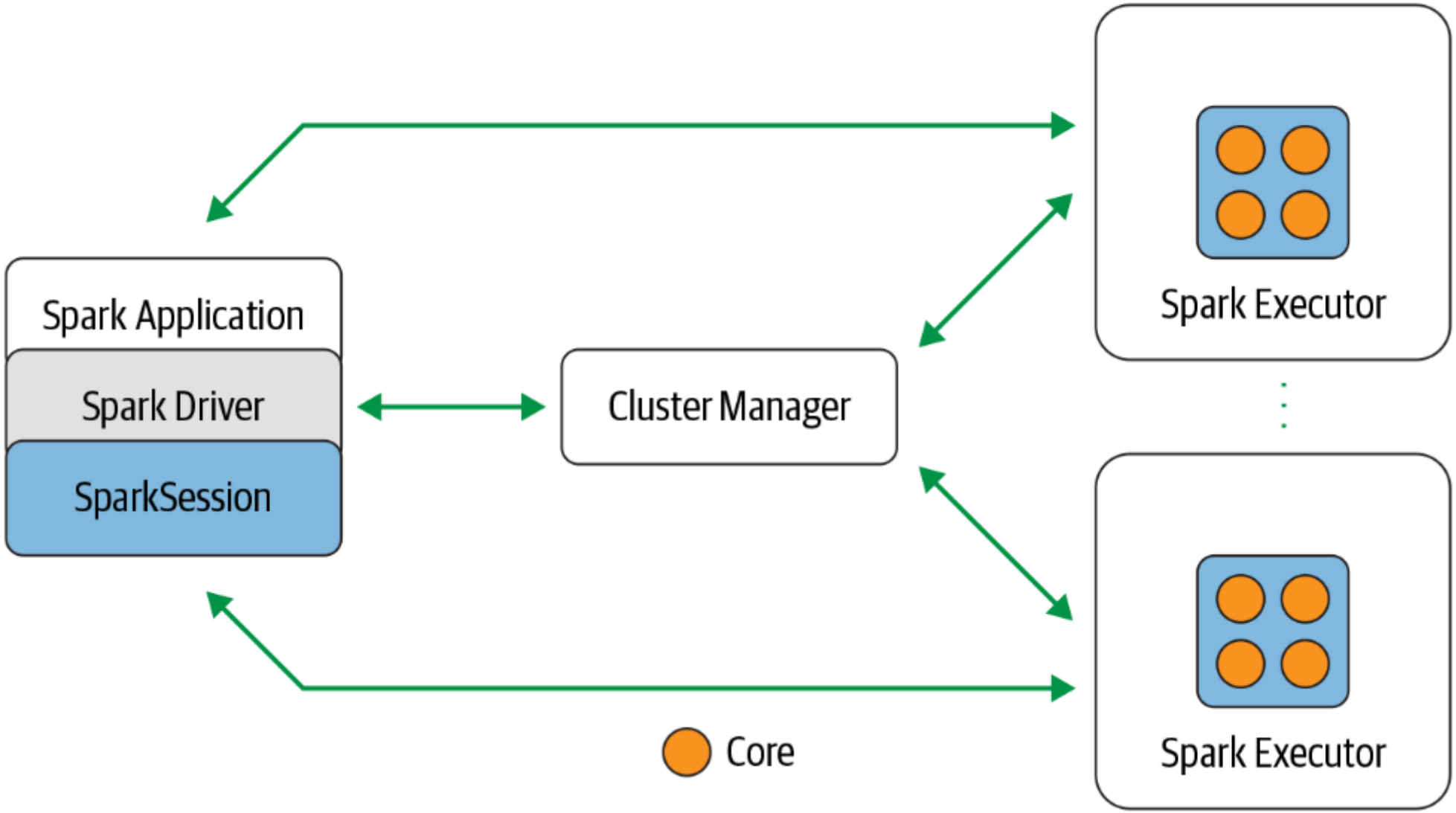

elastic
MySQL
APACHE kafka
And more...
DELTA LAKE
APACHE Spark™
PostgreSQL
redis
hadoop HDFS
mongoDB
APACHE HBASE
cassandra


hadoop
vs
Spark
hadoop
Map Reduce
Spark
YARN
Mesos
hadoop
HDFS
Tachyon
Spark
SQL
mahout
Spark
MLlib
STORM
Spark
Streaming


Submit the code (jar files) and configured dependencies to Executors for further execution
Driver Program
spark = SparkSession.builder..
spark.sparkContext
rdd = spark.read.textFile..
rdd.filter(...)
rdd.map
rdd.count
Action
Request for worker nodes/Executers in the cluster
Cluster Manager
Allocate the resources and instruct workers to execute the job. Tracks submitted jobs and report back the status of jobs.
Worker Node
Executer
Task
Task
Worker Node
Executer
Task
Task
Worker Node
Executer
Task
Task
SparkContext
DAG Scheduler
Task Scheduler
As the Action encounters, it will create the job
Job
RDD1
RDD2
RDD
RDD
Task
Task
Task
Task
Tasks
Launch tasks via cluster manager
Build orerator DAG and Split graph into stages of task and submit each stage as ready to Task Scheduler

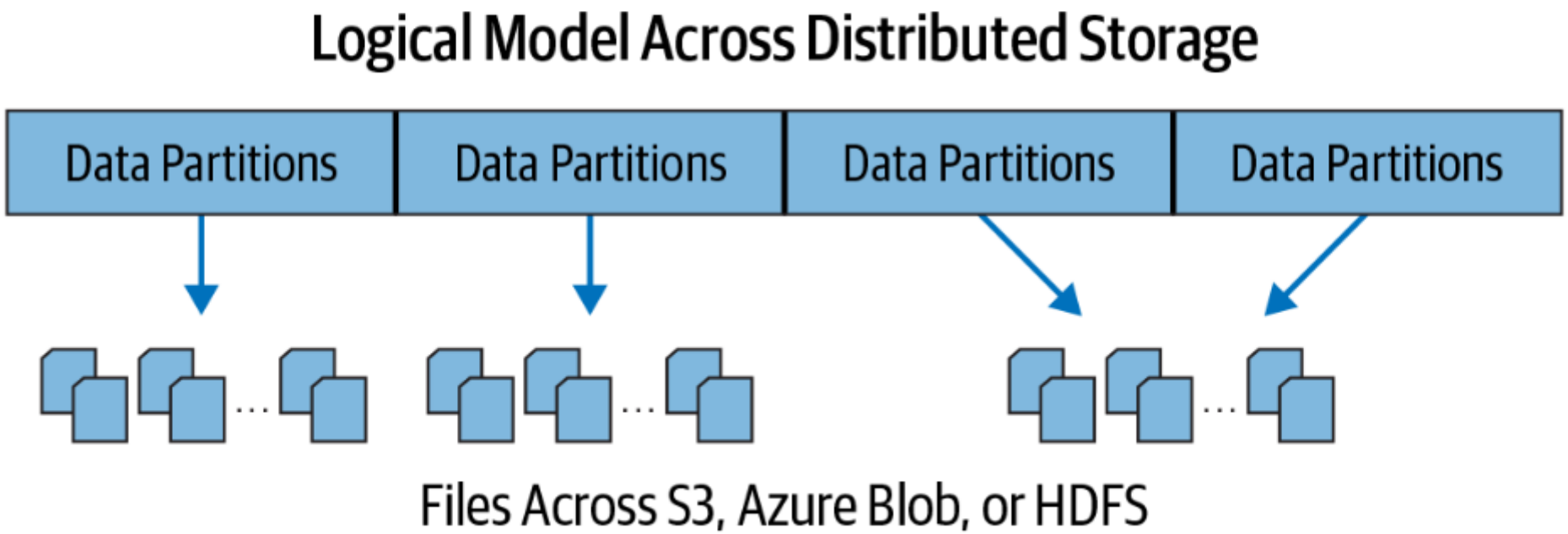
User submits job → SparkContext → Cluster Manager → Worker Nodes → Executors → Task Execution → Result

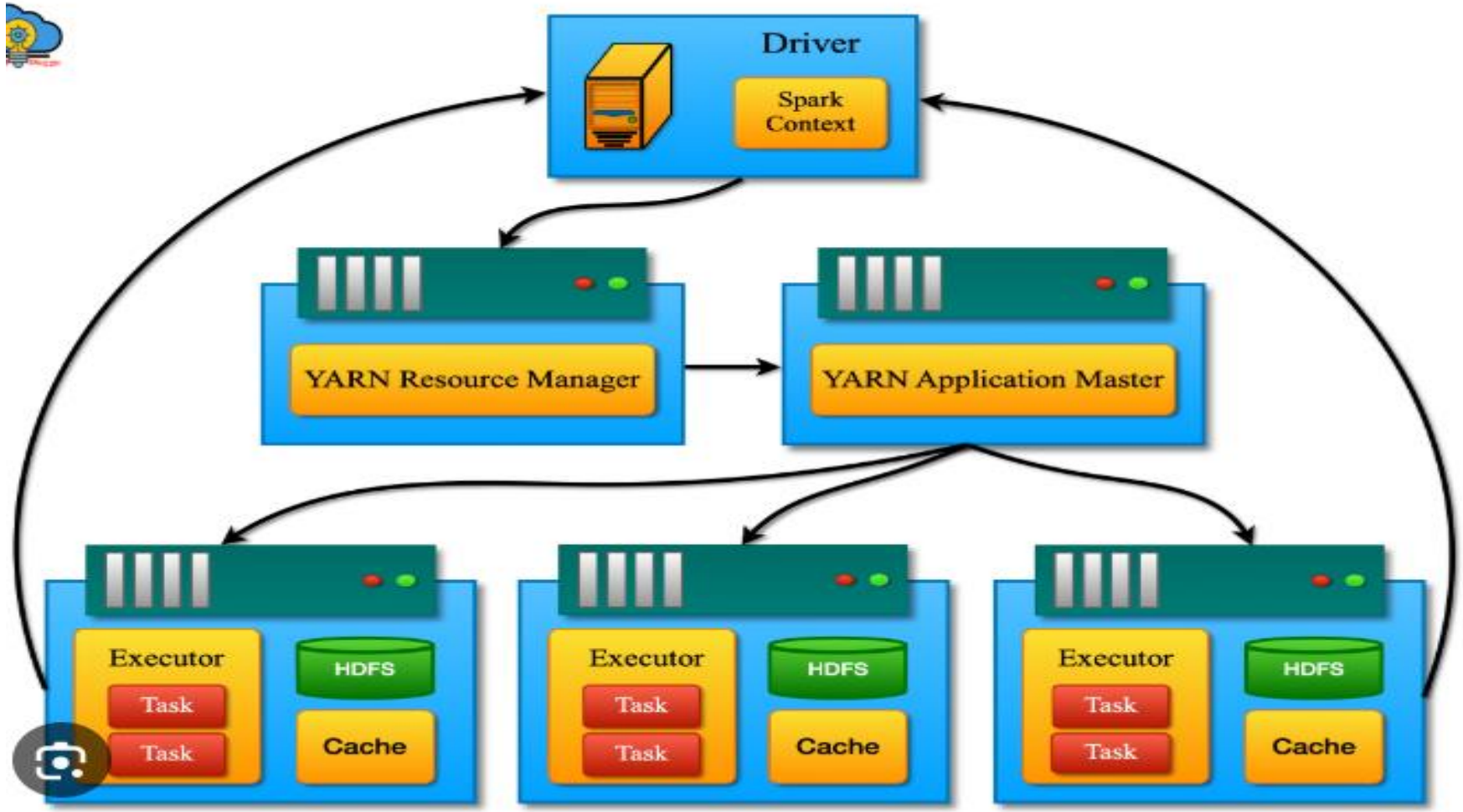

Spark SQL and DataFrames + Datasets
Spark Streaming (Structured Streaming)
Machine Learning MLlib
Graph Processing Graph X
Spark Core and Spark SQL Engine
Scala
SQL
Python
Java
R

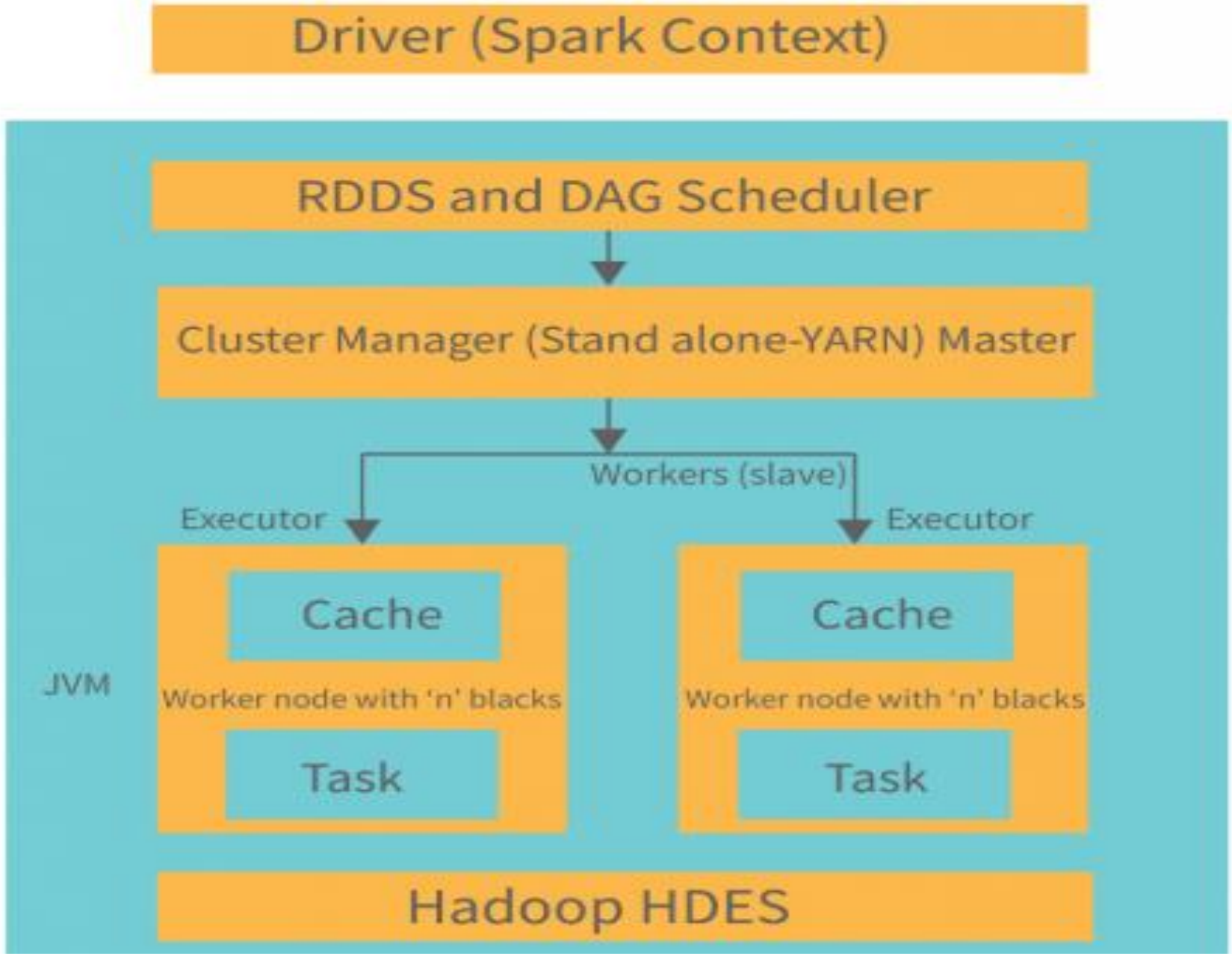

Spark Application
Spark Driver
SparkSession
Cluster Manager
Spark Executor
Spark Executor
Core

Distributed data and partitions

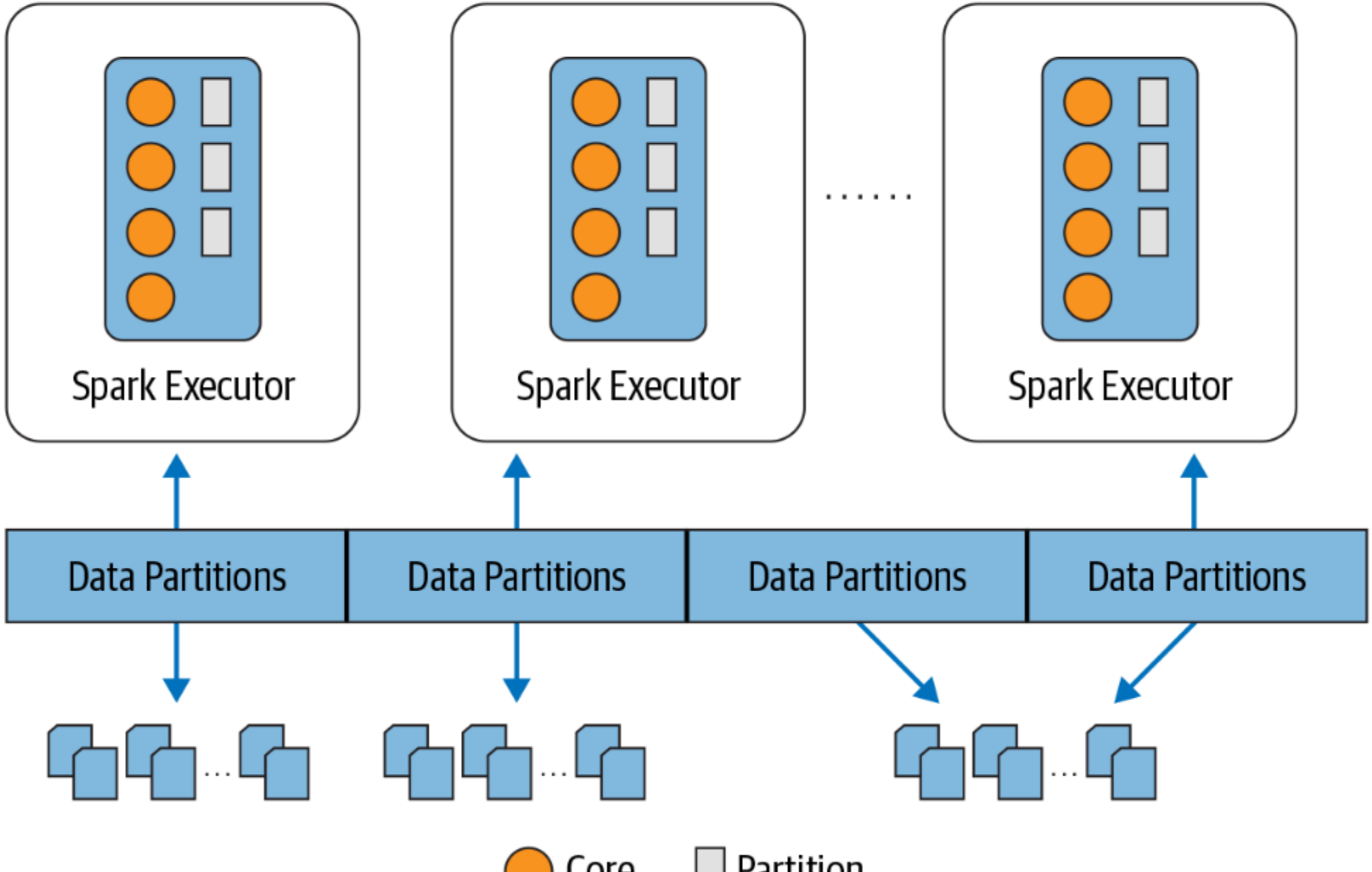
Actual physical data is distributed across storage as partitions residing in either HDFS or cloud storage (see Figure 1-5). While the data is distributed as partitions across the physical cluster, Spark treats each partition as a high-level logical data abstraction—as a DataFrame in memory. Though this is not always possible, each Spark executor is preferably allocated a task that requires it to read the partition closest to it in the network, observing data locality.



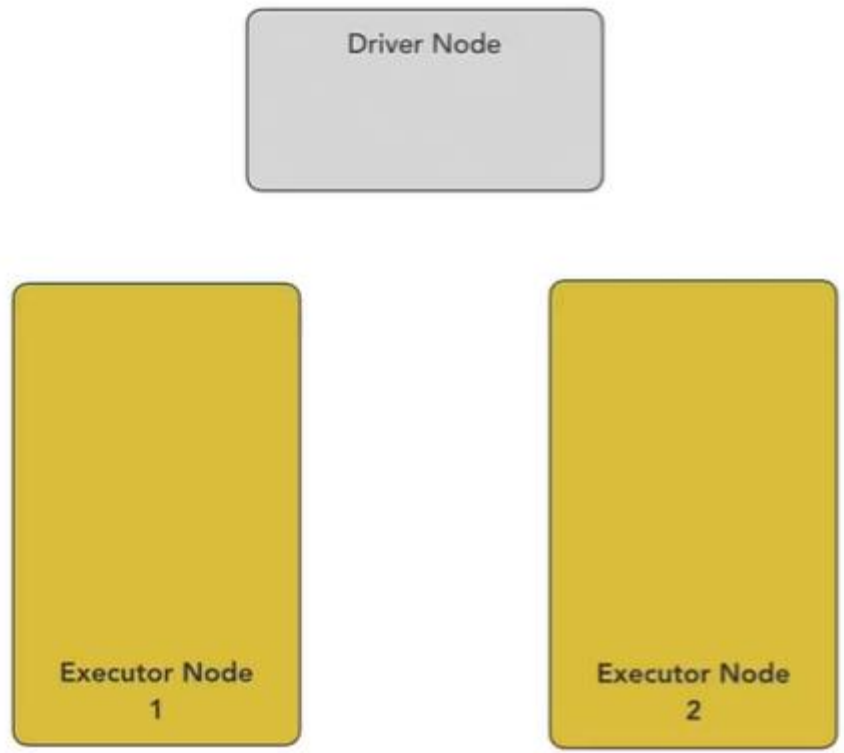

Driver
Spark Context
YARN Resource Manager
YARN Application Master
Executor
Task
Task
HDFS
Cache
Executor
Task
Task
HDFS
Cache
Executor
Task
Task
HDFS
Cache


Driver (Spark Context)
RDDS and DAG Scheduler
Cluster Manager (Stand alone-YARN) Master
Workers (slave)
Executor
Executor
Cache
Cache
JVM
Worker node with 'n' blacks
Worker node with 'n' blacks
Task
Task
Hadoop HDES

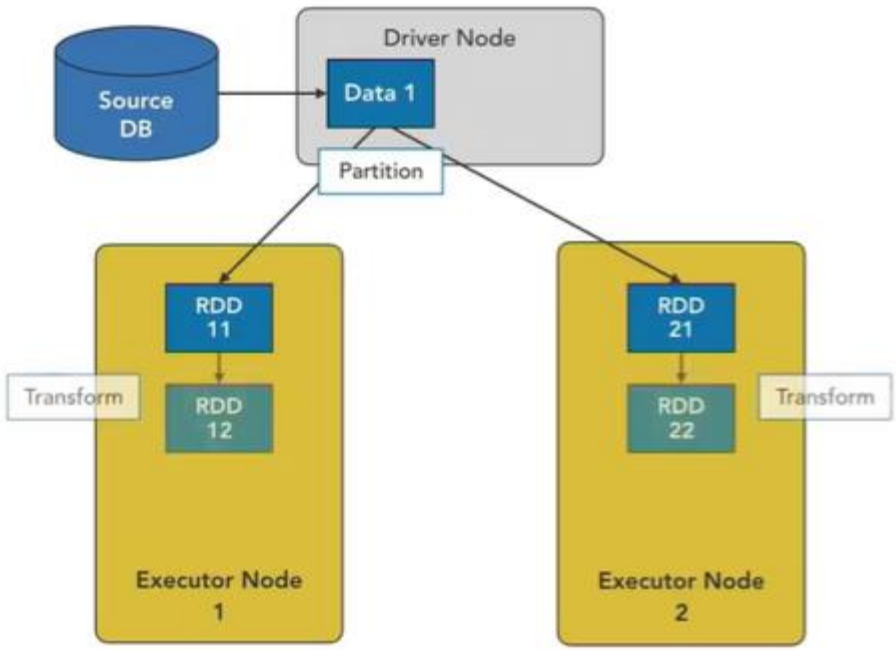
Partitioning allows for efficient parallelism. A distributed scheme of breaking up data into chunks or partitions allows Spark executors to process only data that is close to them, minimizing network bandwidth. That is, each executor's core is assigned its own data partition to work on (see Figure 1-6).



Spark Architecture and Execution

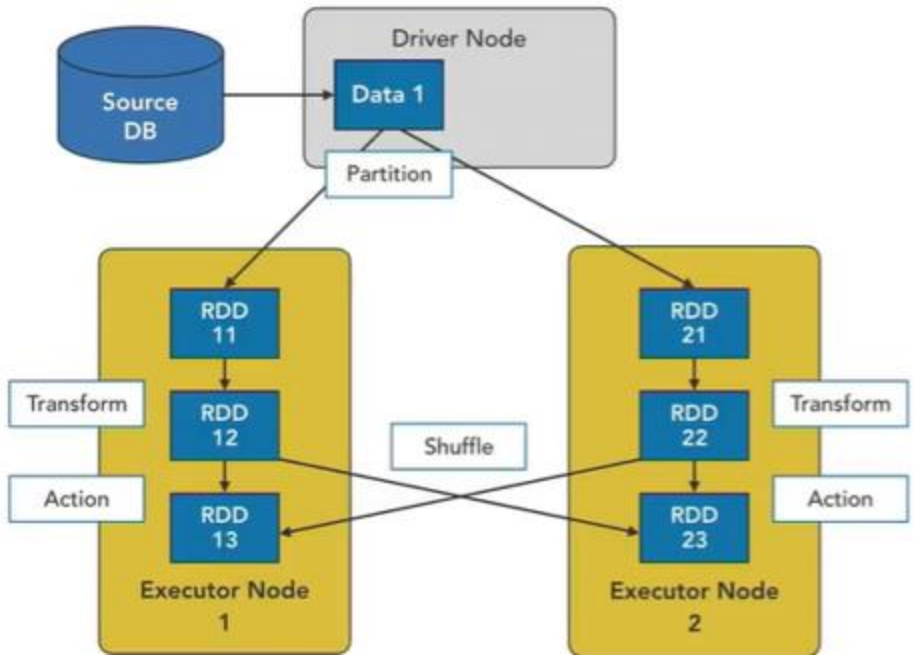


Spark Architecture and Execution



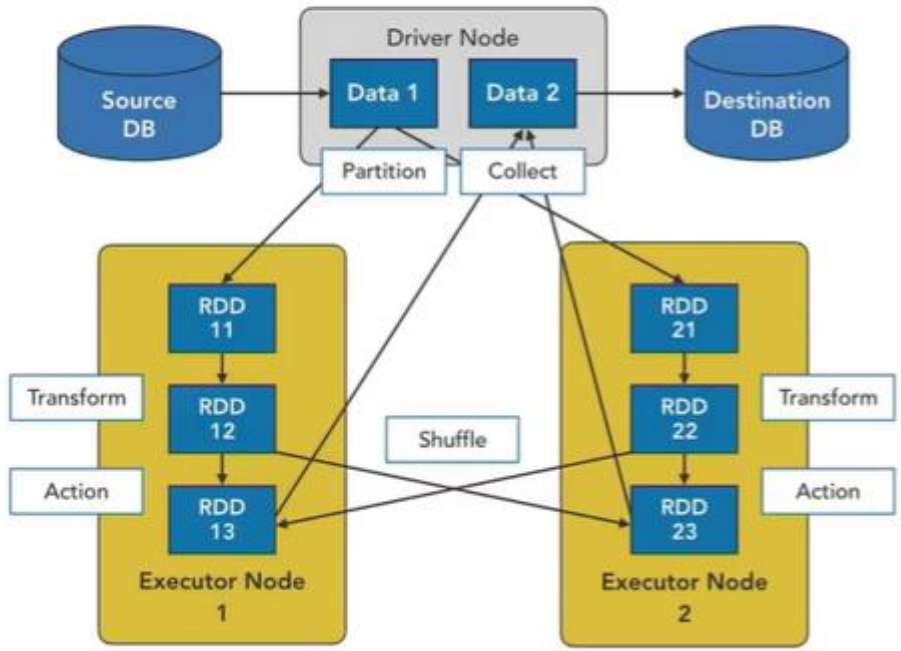
1. Driver node reads Data 1.
2. Data 1 is broken into partitions (RDDs) and distributed to executor nodes.
3. Transform operation happens locally.

Spark Architecture and Execution



1. Driver node reads Data 1.
2. Data 1 is broken into partitions (RDDs) and distributed to executor nodes.
3. Transform operation happens locally.
4. Action operation creates shuffles.

Spark Architecture and Execution



1. Driver node reads Data 1.
2. Data 1 is broken into partitions (RDDs) and distributed to executor nodes.
3. Transform operation happens locally.
4. Action operation creates shuffles.
5. Driver node collects back results.

RDD (Resilient Distributed Dataset) in Apache Spark

RDD is the **fundamental data structure** in Apache Spark. It is an **immutable** (read-only) collection of objects that are **distributed** across multiple nodes in a cluster and **processed in parallel**.

Where Does RDD Run?

RDD runs **on the cluster nodes** of a Spark application. Spark operates in a **master-slave architecture**, where:

- The **Driver Program** runs on the master node.
- The **Executors** run on the worker nodes.
- RDDs are distributed across the worker nodes and are processed in parallel.

Who Initiates an RDD?

RDDs are **initiated by the user** through:

1. **Loading data from an external source** (like HDFS, S3, a database, or local files).
2. **Creating RDDs manually** (using `parallelize()` in Spark).
3. **Transforming existing RDDs** using functions like `map()`, `filter()`, `reduceByKey()`, etc.

Where is RDD Stored?

RDDs can be stored in **different storage levels**, such as:

1. **Memory (RAM)** → Default storage in Spark (`MEMORY_ONLY`).
2. **Disk** → If there is not enough memory, Spark stores RDDs on disk (`DISK_ONLY`).
3. **Both Memory and Disk** → Partial storage in memory and rest on disk (`MEMORY_AND_DISK`).
4. **Serialized format** → Compressed storage to save memory (`MEMORY_ONLY_SER`).

RDD storage is managed by Spark's **caching and persistence mechanisms**.

What Is Hadoop?

Open source

Distributed storage and computing

Can scale to petabytes of data

Consists of:

- Hadoop Distributed File System (HDFS)
- MapReduce

HDFS: Current State

- Still a very good option for cheap storage of large quantities of data
- Suitable for enterprises with in-house data centers
- Cloud alternatives like Amazon S3, Google Cloud Storage, Azure Blob

MapReduce: Current State

- Scales horizontally
- Very slow as it uses disk for data storage
- Being replaced by faster alternatives like Apache Spark

What Is Apache Spark?

- Open source
- Large-scale distributed data processing engine
- Uses memory to speed up computations
- Batch, streaming, ML, and graph capabilities

What Is Apache Spark?

- Scala, Java, Python, and R support
- The most popular big-data processing platform today

The Power of Spark and Hadoop

HDFS provides large-scale distributed storage

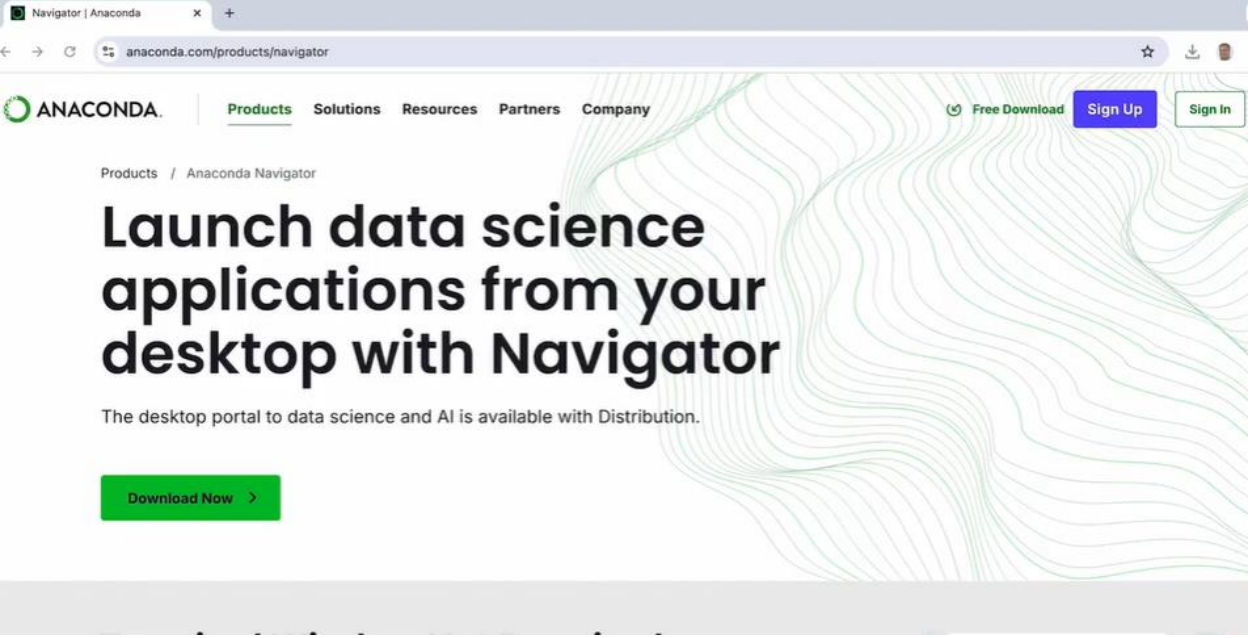
Spark provides large-scale fast processing

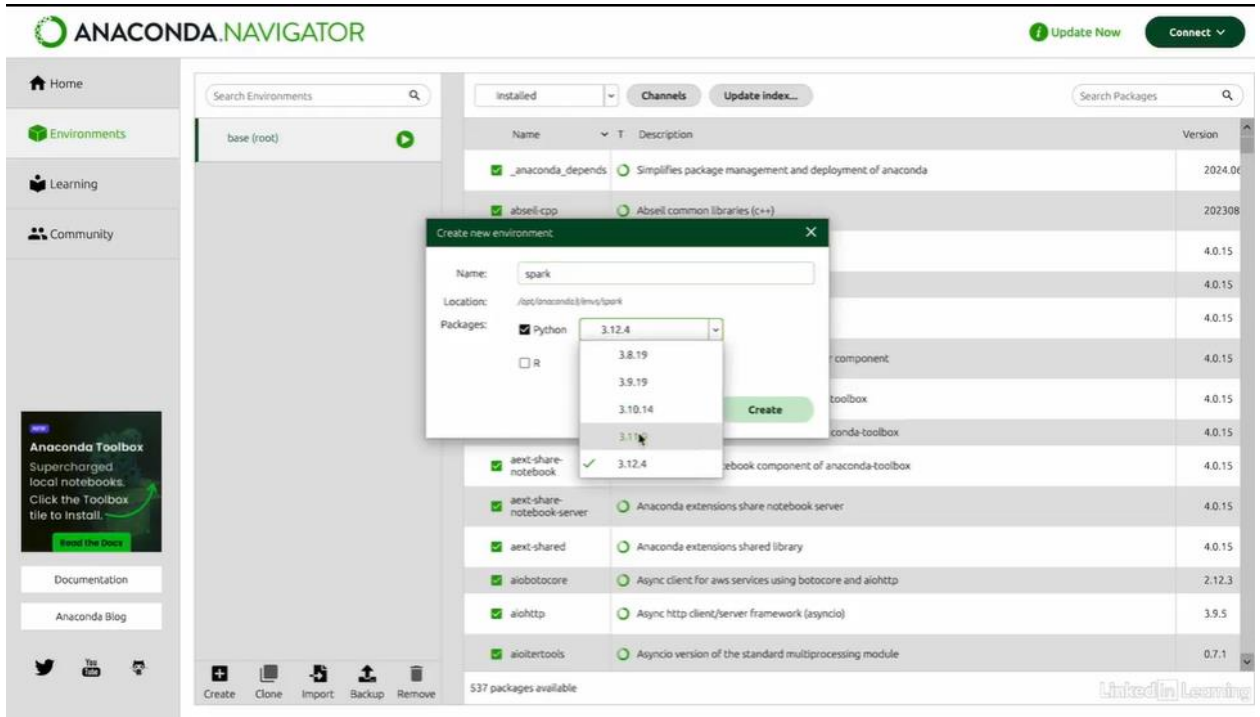
Spark is well integrated with Hadoop

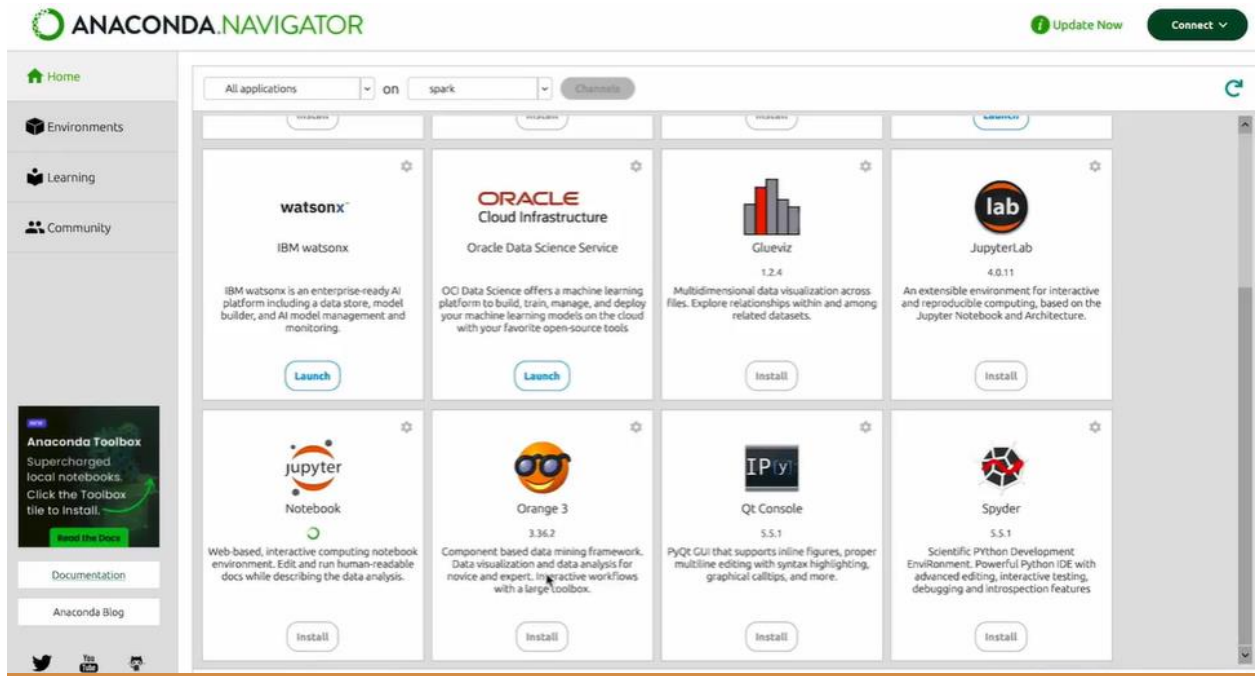
- Parallelism
- Optimizations
- Caching
- YARN

Powerful Combination

Utilize the power of Spark processing over HDFS storage to build scalable, high-performance processing jobs.


Navigator | Anaconda
anaconda.com/products/navigator
ANACONDA.
Products
Solutions
Resources
Partners
Company
Free Download
Sign Up
Sign In
Products / Anaconda Navigator
Launch data science applications from your desktop with Navigator
The desktop portal to data science and AI is available with Distribution.
Download Now


ANACONDA.NAVIGATOR
Update Now
Connect
Home
Environments
Learning
Community
Search Environments
base (root)
Installed
Channels
Update index...
Search Packages
Name
T
Description
Version
_anaconda_depends
Simplifies package management and deployment of anaconda
abseil-cpp
Abseil common libraries (c++)
Create new environment
Name:
spark
Location:
Packages:
Python
3.12.4
R
3.8.19
3.9.19
3.10.14
3.12.4
Create
aext-share-notebook
aext-share-notebook-server
Anaconda extensions share notebook server
aext-shared
Anaconda extensions shared library
aiobotocore
Async client for aws services using botocore and aiohttp
aiohttp
Async http client/server framework (asyncio)
aioitertools
Asyncio version of the standard multiprocessing module
Anaconda Toolbox
Supercharged local notebooks. Click the Toolbox tile to install.
Documentation
Anaconda Blog
Create
Clone
Import
Backup
Remove
537 packages available
LinkedIn Learning



Text Files

- Simple to read/write
- Low performance
- More storage
- No schema

Avro

- Language-neutral data serialization
- Row format
- Self-describing schema support
- Compressible
- Splittable

Parquet

- Columnar format
- Read-only selected columns
- Schema support
- Compressible (column level) and splittable
- Supports nested data structures

Hadoop Storage Formats

- Raw text (blobs)
- Structured text files (CSV, XML, JSON)
- Sequence files
- Avro
- ORC
- Parquet

Parquet for Analytics

Parquet provides overall better performance and flexibility for analytics applications.

Hadoop Compression Options

- Snappy
- LZO
- GZIP
- bzip2

LZO

- Moderate compression
- Excellent processing performance
- Splittable
- Requires separate license

GZIP

- Very good compression
- Moderate processing performance
- Not splittable
- Ideal for container-type applications

bzip2

- Excellent compression
- Slower processing performance
- Splittable
- Ideal for archival type applications

Choosing a Compression Format

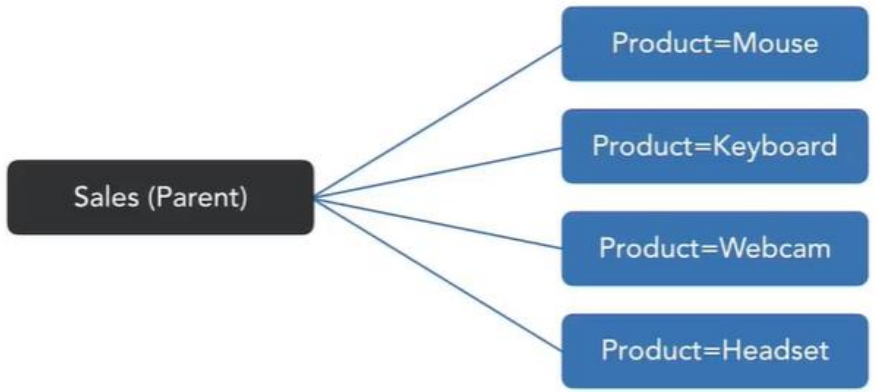
Choose a format based on the application. Do proof of concepts to understand actual performance and storage impacts.

Need for Partitioning

- HDFS does not have the concept of indexes
- Even for reading one row, the entire file should be read
- Partitioning provides a way to read only a subset of data
- Multiple attributes can be used for hierarchical partitioning

Partitioning

- Split data into directories based on individual values of attributes



Choosing Partitions

Choose attributes with a limited list of values and those that are most used in SELECT filters.

Bucketing

- Similar to partitioning
- Subdirectories based on a hash value are generated based on an attribute
- Controls the number of unique directories created
- Even distribution
- Ideal for attributes with large number of unique values

Choosing Buckets

Choose attributes with a large number of unique values and those that are most used in SELECT filters.

Storage Best Practices

- Understand the most used read and write patterns for your data
- Determine what needs optimization and what can be compromised
- Choose options carefully as they cannot be easily changed
- Run tests on actual data to understand performance and storage characteristics
- Choose partitioning and bucketing keys wisely

Data Ingestion Best Practices

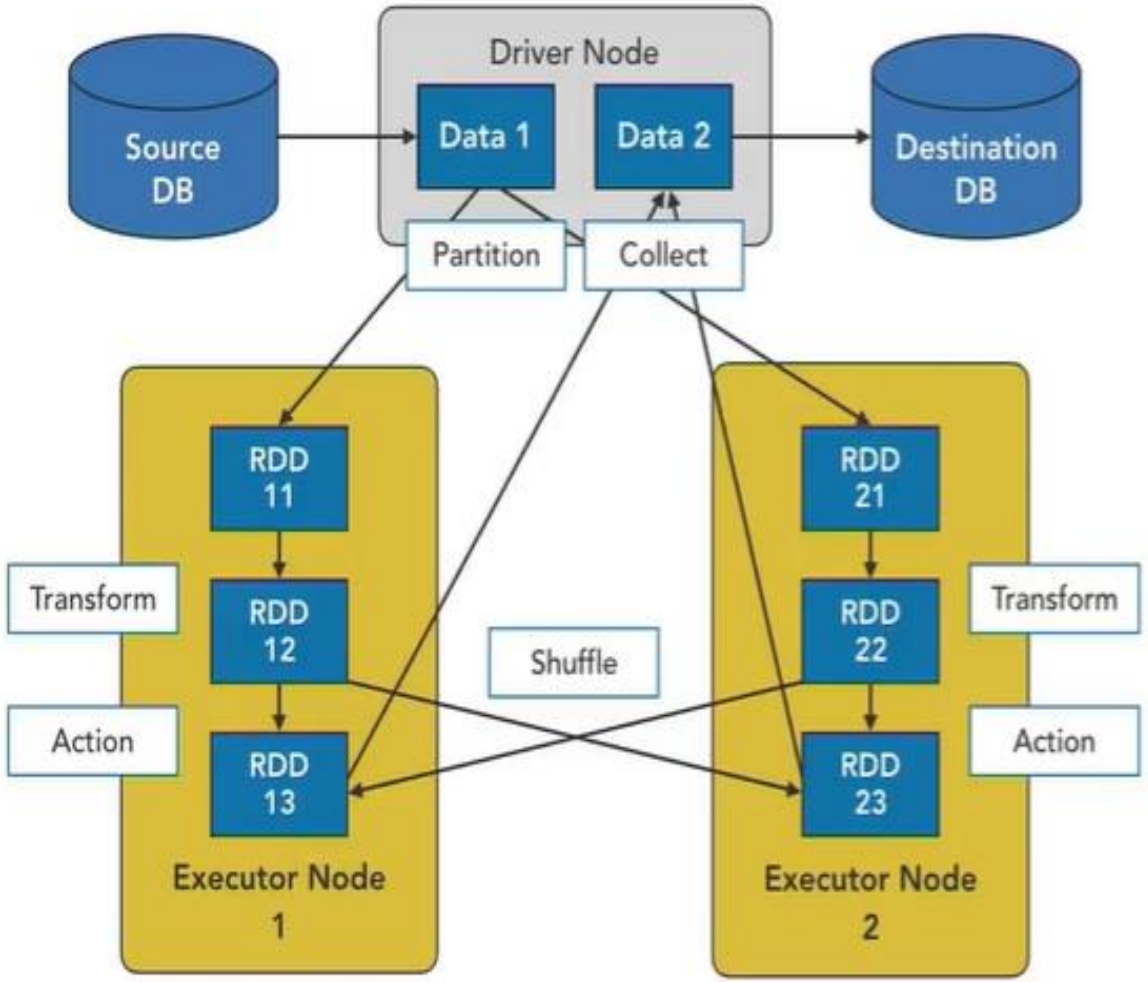
Enable parallelism for maximum write performance

Use APPEND for incremental data ingestion

External data reads – use sources that provide parallelism

- JDBC
- Kafka

Spark Architecture and Execution



1. Driver node reads Data 1.
2. Data 1 is broken into partitions (RDDs) and distributed to executor nodes.
3. Transform operation happens locally.
4. Action operation creates shuffles.
5. Driver node collects back results.

Spark Execution Plan

Lazy execution – only an action triggers execution

Spark optimizer comes up with a physical plan

Physical plan optimizes for:

- Reduced I/O
- Reduced shuffling
- Reduced memory usage

Spark Execution Plan

Spark executors can read and write directly from external sources when they support parallel I/O (for example, HDFS, Kafka, JDBC)

Data Extraction Best Practices

Reduce data read into memory by:

- Using filters based on partition key
- Reading only required columns

Use data sources and file formats that support parallelism

Keep number of partitions:

>= (Number of executors x Number of cores per executor)